

PROMPTS Y RESPUESTAS DE LA IA GENERATIVA - P1



UNIVERSIDAD DE GRANADA

Jaime Corzo Galdó
Técnicas de los Sistemas Inteligentes
2025/26

Prompt: En el ejemplo del guion, no entiendo de donde salen las 14 acciones NIL, entiendo que primero se transforma en murciélago, y luego recorre 6 casillas hacia abajo, pero las acciones NIL que faltan explícame como funcionan.

Respuesta:

Para entender de dónde salen exactamente las 14 acciones ACTION_NIL de ese tramo final, tenemos que repasar las reglas que establece el enunciado sobre el proceso de uso de una catapulta: 1 NIL por cada transformación (de vampiro a murciélago y de murciélago a vampiro). 1 NIL por cada casilla que se avance. 1 NIL adicional al sobrepasar una nueva catapulta mientras se está volando. Si aplicamos estas reglas paso a paso al segundo vuelo del avatar, la cuenta matemática de los 14 ACTION_NIL se desglosa así: Transformación inicial: Al subirse a la catapulta, el avatar pasa de vampiro a murciélago = 1 NIL. Viaje hacia abajo: Avanza 6 casillas hasta llegar a la catapulta inferior = 6 NIL. Primera catapulta en vuelo: Al sobrepasar la catapulta inferior (que actualiza su dirección a la derecha), se consume = 1 NIL. Avance a la derecha: Se desplaza una casilla a la derecha = 1 NIL. Segunda catapulta en vuelo: Llega a otra catapulta en su camino y la sobrepasa = 1 NIL. El tramo final (los "otros 4"): El texto indica literalmente que hay "otros 4 ACTION_NIL más que nos permitirían sobrevolar una casilla con agua, recoger la llave y atravesar el portal". Estos últimos 4 se consumen aplicando las reglas de movimiento y transformación: Avanzar sobre la casilla de agua (1 NIL). Avanzar sobre la casilla de la llave (1 NIL). Avanzar hacia la casilla del portal y la transformación final de murciélago a vampiro (2 NIL). Total: $1 + 6 + 1 + 1 + 1 + 4 = 14$ acciones NIL.

Prompt:

/**

- * Returns a list of observations of portals in the game. As there can be
- * portals of different type, each entry in the array corresponds to a sprite type.
- * Every ArrayList contains a list of objects of type Observation. Each Observation
- * holds the position, unique id and sprite id of that particular sprite.

*

- * @return Observations of portals in the game.

*/

```
public ArrayList<Observation>[] getPortalsPositions() {  
    return model.getPortalsPositions(null);  
}
```

tengo esta función, y quiero obtener las coordenadas del único portal del juego, pero no sé cómo acceder a ellas

Respuesta:

Para obtener las coordenadas de ese único portal, el principal reto es que, aunque solo haya uno, no sabemos de antemano de qué "tipo de sprite" es. Por lo tanto, no

sabemos en qué índice del array (el `ArrayList<Observation>[]`) se encuentra guardado.

La forma más segura de obtenerlo es recorrer el array hasta encontrar la lista que no esté vacía, extraer el primer (y único) elemento de esa lista, y finalmente leer su posición.

Prompt:

Para obtener donde están las catapultas y las monedas al inicio, como sería, hay alguna manera de obtener el mapa del juego y recorriéndolo o con algún método de the state observation como como se haría eso. Ya que una vez que accedo al mapa como distingo entre lo que es moneda, llave, catapulta, etc.

Respuesta:

Tu fichero catapults2026.txt te lo dice bastante claro.

En SpriteSet y LevelMapping aparece que:

c y m son monedas (coins),
k es llave (key),
g es la puerta/meta cerrada (locked),
w es muro/planta (wall),
. es agua (water),
0 es catapulta este,
1 es catapulta sur,
2 es catapulta norte,
3 es catapulta oeste,
A es el avatar inicial,
_ es suelo.

El problema real: dentro de resources o immovables, ¿qué sublista es cada cosa?

Ahí entra itype.

Porque el API te agrupa por tipo, pero no te pone el nombre del tipo en texto. Te pone un itype entero. Observation solo guarda category, itype, obsID y position.

Entonces, la forma más práctica de trabajar es:

imprimir una vez al principio todos los tipos que te salen,
mirar en el mapa/visualización cuál es cuál,
guardar esos itype en variables del agente.

Lo importante: no intentes deducir todo solo desde getObservationGrid()

Puedes hacerlo, pero para empezar te complica.

Porque en este juego muchas casillas tienen más de un sprite. Por ejemplo, en el LevelMapping:

```
c > coins floor  
m > coins water  
w > wall floor  
0 > eastfacing floor  
g > locked floor
```

Eso significa que en una misma celda puede haber varias observaciones: suelo + moneda, o agua + moneda, o suelo + catapulta, etc. Así que si empiezas directamente por el grid bruto, te lías más.

Prompt:

como hago para añadir la metrica de tiempo en milisegundos? No tanto para añadirla si no para calcular el tiempo de ejecucion en java.

Respuesta:

Sí puedes añadirlo, porque MetricsProvider tiene setTiempoMilisegundos(long) y también lo imprime. La forma más simple es medir solo la búsqueda, el cálculo del plan. En java lo puedes hacer así:

```
long tInicio = System.nanoTime();  
  
// Búsqueda  
  
long tFin = System.nanoTime();  
tiempoMilisegundos = (tFin - tInicio) / 1_000_000L;
```

Prompt:

para un algoritmo de busqueda en profundidad en java, uso ArrayList<Estado> para almacenar los visitados, no se si este es el motivo de que sea ineficiente, ya que me tarda mas tiempo del que debería, que otras alternativas de estructuras de datos hay para almacenar los nodos visitados de una manera mas eficiente?

Respuesta:

Tienes toda la razón en tu sospecha. Usar un ArrayList para almacenar los nodos visitados es exactamente el motivo por el que tu algoritmo de búsqueda en

profundidad (DFS) se ha vuelto ineficiente. El problema radica en cómo funciona el método `.contains()` en un `ArrayList`. Para saber si un estado ya fue visitado, Java tiene que recorrer la lista elemento por elemento desde el principio hasta encontrarlo (o hasta llegar al final). Esto tiene una complejidad de tiempo de $O(N)$. A medida que tu algoritmo explora más nodos, la lista crece y cada comprobación se vuelve exponencialmente más lenta. Aquí tienes las mejores alternativas en Java, ordenadas de la más común a la más optimizada según tu caso de uso. La solución ideal: `HashSet<Estado>` Esta es la alternativa directa y más utilizada. Un `HashSet` utiliza una tabla hash interna, lo que significa que las operaciones de añadir (`.add()`) y buscar (`.contains()`) tienen una complejidad de tiempo promedio de $O(1)$ (tiempo constante). Tu algoritmo notará una mejora de velocidad drástica. ¡Requisito indispensable! Para que un `HashSet` funcione correctamente, tu clase `Estado` debe sobrescribir los métodos `equals()` y `hashCode()`. Si no lo haces, Java comparará las referencias de memoria de los objetos en lugar de su contenido, y el `HashSet` no detectará correctamente los estados duplicados.

Prompt:

he visto que al tener muchos elementos en mi clase estado, el metodo hasCode deja de ser eficiente ademas de que tiene 3 matrices dinamicas: monedas restantes, catapultasRestantes y llavesRestantes, he visto que una idea seria tener un valor entero que sea el que devuelva hashCode(), pero luego en en la clase de AgenteProfundidad tendria que calcularlo, cual seria la idea para hacerlo, y que funciones se usan para calcular el hashCode de las matrices, y que import deberia incluir?

Respuesta:

Es una observación excelente. Lo que estás planteando se conoce como "Caching del HashCode" (almacenar en caché el hash) y es una técnica de optimización avanzada muy común en Java, especialmente en algoritmos de búsqueda y grafos.

Como tu clase `Estado` actúa casi como un objeto inmutable (una vez que lo creas en `aplicarAccion`, ya no modificas sus valores, sino que creas uno nuevo para el siguiente paso), no tiene sentido recalcularlo el costoso `hashCode()` cada vez que el `HashSet` necesita comprobarlo. ¡Especialmente con tres matrices bidimensionales!

Aquí tienes la idea exacta de cómo implementarlo, qué funciones usar y los imports necesarios:

1. Los Imports Necesarios

```
import java.util.Arrays;
import java.util.Objects;
```

2. La función para matrices bidimensionales

Para calcular el hash de un array de primitivos bidimensional (`boolean[][]`), la función estándar, segura y correcta en Java es `Arrays.deepHashCode(matriz)`. Esta función recorre los elementos internos de la matriz para generar un identificador único basado en su contenido.

3. La Idea: Cómo modificar Estado.java

La idea es añadir una variable entera para guardar el hash, crear un método específico para calcularlo una sola vez, y hacer que el método `@Override hashCode()` simplemente devuelva esa variable.

¿Por qué esto es tan eficiente?

Al hacer esto, cuando el algoritmo DFS coge un nodo, genera sus sucesores y hace `visitados.contains(sucesor.estado)`, el método `hashCode()` se limitará a devolver un `int` de memoria, sin tener que recorrer ni una sola vez las filas y columnas de tus 3 matrices. La reducción de tiempo de CPU en laberintos grandes será gigantesca.

Prompt:

Se sigue pasando de tiempo, en el último mapa, que es el más grande, ya he implementado lo del `hashCode()`, así que lo único que se me ocurre son las matrices que tengo dinámicas que cuanto más grande sea el mapa más grande van a ser, hay alguna estructura de datos que puedo usar para sustituirlas, manteniendo lo del `hashCode()`?

Respuesta:

Sí: lo he revisado bien y la conclusión es bastante clara. El problema es que el `HashSet` no está atacando el cuello de botella real de tu implementación actual. En tu código, cada estado nuevo termina llamando a `nuevo.calcularHash()`, y ese método recorre completas las tres matrices dinámicas con `Arrays.deepHashCode(...)`. Además, cuando recoges objetos o desactivas catapultas, sigues copiando matrices completas. Eso hace que el coste fuerte siga siendo “recorrer/copiar el mapa” por cada sucesor, no la estructura de `visitados`.

El verdadero problema está en Estado.java y en cómo lo usa AgenteProfundidad:

`calcularHash()` sigue haciendo un hash profundo de tres `boolean[][]`, así que cada estado nuevo vuelve a escanear mucho contenido.

`equals()` también compara completas esas matrices si hay colisión.

`recogerObjetos(...)` y la lógica de catapultas siguen copiando matrices enteras cuando hay cambios.

La solución de verdad

Para que baje el tiempo de forma seria, yo ya no seguiría optimizando `HashSet` + matrices.

La solución razonable es cambiar la representación dinámica del estado:

En vez de guardar esto en Estado

`boolean[][] monedasRestantes`

`boolean[][] catapultasRestantes`

`boolean[][] llavesRestantes`

Guarda esto

`BitSet monedasRestantes`

`BitSet catapultasRestantes`

`BitSet llavesRestantes`

y en el agente guardas unas tablas estáticas de índice:

`int[][] indiceMoneda`

`int[][] indiceCatapulta`

`int[][] indiceLlave`

inicializadas a -1.

Entonces cada moneda, llave y catapulta dinámica tiene un índice pequeño, y el estado solo guarda si ese objeto sigue existiendo o no.

Por qué esto sí acelera

Porque pasarías de trabajar con todo el grid a trabajar solo con los objetos dinámicos reales.

Ahora mismo tu hash y tus copias dependen del tamaño del mapa.

Con BitSet, dependen del número de monedas/catapultas/llaves, que suele ser muchísimo menor.

Mi diagnóstico final

No estás haciendo “mal” el DFS ni “mal” el HashSet.

Lo que pasa es que tu representación del estado sigue siendo demasiado cara para el tiempo límite. En el nivel 7, el coste dominante sigue estando en el manejo de `boolean[][]`, no en la estructura de visitados. Tu código actual ya muestra claramente que el hash se recalcula tras cada transición y que las matrices aún se copian en cambios dinámicos.

Así que mi recomendación firme es esta:

deja de optimizar la versión con matrices y cambia ya a una representación compacta de objetos dinámicos con índices + BitSet.

Es el cambio que más probabilidades tiene de bajarte de verdad de ~100 ms a algo competitivo.

Prompt:

tengo este comentario para la cabecera de la clase de AgenteProfundidad, ayudame a escribirlo mejor y mejorarlo:

/**

- * Clase que modeliza el algoritmo de búsqueda en profundidad
- * QUE HACE
- * Calcula el plan que debe seguir el vampiro para llegar a la meta, lo hace
- * utilizando la busqueda en profundidad, que es no informada, lo que quiere decir que no usa
- * informacion heuristica.
- * COMO LO HACE
- * Para calcular el plan, usamos una representacion de los nodos de la busqueda, que incluyen el estado correspondiente,
- * que conforme avanza el algoritmo se van modificando según el mapa. Se ha seguido el pseudocodigo proporcionado, con
- * la unica diferencia de que en vez de generar todos los sucesores y luego recorrer esa lista, se va haciendo directamente
- * la exploracion en el orden correspondiente, pero la logica sigue siendo la misma.
- * Además se ha usado una estructura HashSet para almacenar los nodos visitados, y para representar los elementos dinamicos
- * del mapa, como son las monedas, llaves y catapultas, se ha optado por BitSet.
- * POR QUÉ
- * Se ha optado por usar HashSet en vez de ArrayList ya que las busquedas en este ultimo son del orden de $O(n)$, mientras que
- * HashSet utiliza una tabla hash (mediante hashCode) permitiendo búsquedas e inserciones en tiempo constante $O(1)$.
- * Por otro lado, representar con matrices booleanas los elementos dinamicos del mundo era super costoso conforme los
- * mapas se iban haciendo mas grandes, y no se conseguía explotar la ventaja que daba usar la tabla hash, por lo que la estructura
- * de BitSet resulta mucho mas eficiente en memoria y para luego clonarla tambien.

*/

Respuesta:

Tu borrador base es realmente bueno porque ya capturas perfectamente la esencia técnica de lo que has hecho y por qué lo has hecho. Identificas los cuellos de botella exactos (el $O(N)$ del ArrayList y el coste de memoria de las matrices).

Para elevarlo a un nivel más académico y profesional, ideal para la rúbrica de evaluación, he pulido la redacción. He eliminado expresiones un poco más coloquiales (como "súper costoso") y he estructurado las justificaciones de forma más esquemática para que el profesor pueda leer de un vistazo todas tus optimizaciones.

Aquí tienes la versión mejorada que puedes copiar directamente en tu clase:

Java

/**

* Clase que modeliza el algoritmo de búsqueda en profundidad (Depth-First Search).

* * QUÉ HACE:

* Calcula el plan de acciones secuencial que debe ejecutar el agente (vampiro/murciélago)

* para alcanzar la meta. Al ser un algoritmo de búsqueda no informada (ciega), explora

* el espacio de estados de forma exhaustiva sin utilizar información heurística sobre

* la distancia al objetivo.

* * CÓMO LO HACE:

* Utiliza una representación basada en Nodos (que guardan la traza y profundidad de la búsqueda)

* y Estados (que representan la configuración del mundo en un momento dado).

Implementa

* el algoritmo DFS de forma recursiva. A diferencia del pseudocódigo clásico, no genera

* ni almacena la lista completa de sucesores en cada paso; en su lugar, genera, valida y

* explora cada sucesor directamente en el orden establecido. Utiliza un conjunto `HashSet`

* para el registro de estados explorados y objetos `BitSet` para monitorizar los elementos

* dinámicos del mapa (monedas, llaves y catapultas).

* * POR QUÉ (Decisiones de diseño y optimización):

* - HashSet frente a ArrayList: En DFS, el registro de estados visitados crece exponencialmente.

* Verificar duplicados en un ArrayList requiere un tiempo de $O(N)$, creando un cuello de botella.

* El HashSet, apoyado en un cálculo eficiente del hashCode, permite búsquedas e inserciones

* en tiempo constante $O(1)$.

* - BitSet frente a boolean[][]: Representar la existencia de elementos dinámicos con matrices

- * bidimensionales resultaba prohibitivo en mapas grandes, tanto en consumo de memoria como
- * en el tiempo necesario para clonaras en cada nuevo estado. `BitSet` comprime esta información
- * a nivel de bits, haciendo que las operaciones de copiado mutación sean drásticamente más rápidas.
- * - Generación de sucesores "al vuelo": Evaluar las acciones directamente sin instanciar
- * una lista temporal de sucesores reduce significativamente la generación de "basura" en
- * memoria (Garbage Collection), acelerando el tiempo total de ejecución.
- */

Prompt:

tengo este comentario para la cabecera de la clase de AStart, ayudame a escribirlo mejor y mejorarlo:

Respuesta:

Tienes una base excelente. Has captado perfectamente la idea de las estructuras duales y las optimizaciones de memoria, lo cual demuestra que entiendes tu propio código (algo que los profesores valoran muchísimo).

Sin embargo, el tono de algunas partes (como "es terrible para buscar cosas") es el que yo usé para explicártelo de forma coloquial. Para una entrega universitaria o profesional, el lenguaje debe ser más técnico, formal y preciso. Además, tenías una pequeña incongruencia donde mencionabas HashSet en el "Por qué", pero en tu código estás usando HashMap.

Aquí tienes una versión pulida, redactada con el rigor técnico que se espera en una evaluación académica:

Java

```
/**
```

```
 * Clase que implementa el agente basado en el algoritmo de búsqueda A* (A-Estrella).
```

```
 * * QUÉ HACE:
```

```
 * Calcula de forma secuencial y determinista el plan de acciones óptimo que debe ejecutar
```

```
 * el agente (alternando entre su forma de vampiro y murciélago) para alcanzar el portal
```

```
 * de salida. Se trata de una búsqueda "offline": el árbol de búsqueda y el plan completo
```

- * se computan en el primer tick de reloj, antes de comenzar la ejecución de las acciones.

- * * CÓMO LO HACE:

- * El algoritmo explora el mapa separando la lógica en "Estados" (que representan la configuración exacta del mundo en un momento dado: posición, inventario, vuelo)

y

- * "Nodos" (que guardan la traza del árbol de búsqueda, el coste acumulado g , la heurística h

- * y la acción que llevó a él).

- * * Para gestionar los nodos descubiertos por explorar ("Abiertos"), emplea una estructura

- * dual: una PriorityQueue y un HashMap. Para los nodos ya expandidos ("Cerrados"), utiliza

- * un segundo HashMap. El proceso iterativo extrae siempre el nodo con menor coste $f(n)$,

- * genera sus sucesores válidos al vuelo y actualiza las estructuras hasta encontrar la meta

- * o vaciar la cola de abiertos.

- * * POR QUÉ (Decisiones de diseño y optimización para el rendimiento):

- * - Estructura dual en Abiertos (PriorityQueue + HashMap): La cola de prioridad es ideal

- * para extraer el nodo más prometedor rápidamente, pero verificar si un estado ya existe

- * en ella o actualizar su coste requiere un tiempo lineal $O(N)$. Al respaldarla con un HashMap,

- * las comprobaciones de existencia y la recuperación de nodos para actualizar rutas más

- * cortas se realizan en tiempo constante $O(1)$, eliminando el principal cuello de botella.

- * - HashMap en Cerrados (frente a listas): Garantiza que verificar si un estado ya fue

- * visitado cueste $O(1)$ en lugar del $O(N)$ que requeriría iterar sobre un ArrayList.

- * - Uso de BitSet (frente a boolean[][] o listas de objetos): El estado del juego requiere

- * saber qué monedas, llaves y catapultas siguen activas. Representar esto con matrices

- * implicaba un alto coste computacional al clonar el estado para cada sucesor.

- * `BitSet`

- * comprime esta información a nivel de bits, permitiendo copias y mutaciones casi instantáneas

- * con un consumo de memoria mínimo.

- * - Generación de sucesores "al vuelo": En lugar de instanciar y devolver una lista

- * temporal de nodos sucesores en cada expansión, los sucesores se evalúan y encolan

- * directamente. Esto reduce drásticamente la creación de "basura" en memoria, aliviando
- * el trabajo del recolector de basura (Garbage Collector) y acelerando el tiempo total.
- */

Prompt:

tengo este comentario para la cabecera de la clase de RTAStart, ayudame a escribirlo mejor y mejorarlo:

/**

* Clase que implementa el agente basado en el algoritmo de búsqueda RTA*.

* * QUÉ HACE

* Se trata de un algoritmo de búsqueda en tiempo real, esto quiere decir que tiene un

* espacio local de búsqueda: el agente calcula rutas cortas hasta la mejor frontera, y

* tiene un espacio local de aprendizaje: se actualiza la heurística de cara a movimientos

* futuros. A diferencia de A*, en cada paso, calcula el mejor nodo al que desplazarse,

* y actualiza el espacio local de aprendizaje.

* * CÓMO LO HACE:

* Para encontrar la mejor acción, se exploran los vecinos del nodo actual, y se escoge al mejor.

* Para ello se hace uso de una función heurística, en nuestro caso, la distancia Manhattan al

* objetivo. El valor de la heurística lo almacenamos en una tabla hash, que en nuestro caso

* es un mapa donde las claves son los estados y los valores son las heurísticas, que se van

* actualizando según la regla de aprendizaje

* * POR QUÉ (Decisiones de diseño y optimización para el rendimiento):

* - Se ha elegido un HashMap para almacenar la heurística de cada estado ya que en primer

* lugar, al tratarse de un mapa, nos permite acceder con facilidad a cada valor, y al usar

* una tabla hash, hace que sea super eficiente en operaciones de acceder y actualizar valores,

* que es el uso que le vamos a dar.

* - Uso de BitSet (frente a boolean[][] o listas de objetos): El estado del juego requiere

* saber qué monedas, llaves y catapultas siguen activas. Representar esto con matrices

* implicaba un alto coste computacional al clonar el estado para cada sucesor.

`BitSet`

- * comprime esta información a nivel de bits, permitiendo copias y mutaciones casi instantáneas
- * con un consumo de memoria mínimo.
- * - Generación de sucesores "al vuelo": En lugar de instanciar y devolver una lista temporal de nodos sucesores en cada expansión, los sucesores se evalúan y encolan
- * directamente. Esto reduce drásticamente la creación de "basura" en memoria, aliviando
- * el trabajo del recolector de basura (Garbage Collector) y acelerando el tiempo total.
- */

Respuesta:

/**

- * Clase que implementa el agente basado en el algoritmo de búsqueda heurística en tiempo real RTA* (Real-Time A*).
- * * * QUÉ HACE:
- * Guía al avatar hacia el objetivo tomando decisiones paso a paso (online). A diferencia de
- * algoritmos offline como A* (que calculan todo el plan antes de ejecutar el primer movimiento),
- * RTA* calcula solo la siguiente acción basándose en un espacio local de búsqueda. Además,
- * incorpora un espacio local de aprendizaje: actualiza la heurística de los estados visitados
- * para rellenar "valles" de error, escapar de mínimos locales y evitar bucles infinitos.
- * * * CÓMO LO HACE:
- * En cada iteración (tick del juego), el agente genera los sucesores inmediatos del estado actual.
- * Evalúa cada sucesor sumando el coste del paso y su valor heurístico (calculado inicialmente
- * mediante la distancia Manhattan al objetivo). Selecciona el sucesor con el menor coste total y
- * actualiza el valor heurístico del estado actual asignándole el valor del *segundo* mejor
- * sucesor (regla de aprendizaje de RTA*). Los valores heurísticos actualizados se almacenan y
- * recuperan utilizando una tabla Hash (HashMap).
- * * * POR QUÉ (Decisiones de diseño y optimización para el rendimiento):
- * - Uso de HashMap para la heurística: Permite consultar y actualizar los valores aprendidos de
- * los estados en tiempo constante $O(1)$, minimizando la sobrecarga computacional durante la
- * búsqueda.

- * - Variables de estado con BitSet (frente a boolean[][] o listas): El entorno requiere saber
 - * qué monedas, llaves y catapultas siguen activas. Representar esto con matrices implicaría
 - * un alto coste al clonar el estado para simular cada acción. BitSet comprime esta información
 - * a nivel de bits, permitiendo copias y mutaciones casi instantáneas con un consumo de memoria mínimo.
- * - Evaluación de sucesores "al vuelo": En lugar de instanciar y almacenar temporalmente los
 - * nodos sucesores en estructuras de datos complejas (como listas o colas), los vecinos se
 - * simulan, se evalúan frente a los mejores valores registrados y se descartan inmediatamente.
 - * Esto reduce drásticamente la creación de objetos innecesarios en memoria, aliviando el
 - * trabajo del recolector de basura (Garbage Collector) y acelerando el tiempo total de ejecución.

*/

Prompt:

tengo este comentario para la cabecera de la clase de LRTAStarK, ayudame a escribirlo mejor y mejorarlo:

```
/**
 * Clase que implementa el agente basado en el algoritmo de búsqueda heurística
 * en tiempo real LRTA*(k) .
 * * * QUÉ HACE:
 * Guía al avatar hacia el objetivo tomando decisiones paso a paso (online). A
 * diferencia de
 * algoritmos offline como A* (que calculan todo el plan antes de ejecutar el primer
 * movimiento),
 * LRTA* calcula solo la siguiente acción basándose en un espacio local de
 * búsqueda. Además,
 * incorpora un espacio local de aprendizaje: actualiza la heurística de los estados
 * visitados
 * para rellenar "valles" de error, escapar de mínimos locales y evitar bucles infinitos.
 * La única diferencia entre RTA* y LTRA* es la regla de aprendizaje, ya que se usa
 * el segundo
 * mínimo y el primer mínimo respectivamente. La ventaja que tiene LRTA* es que en
 * múltiples ejecuciones
 * converge más rápido al óptimo. Además, LRTA*(k) incluye la propagación de los
 * cambios heurísticos
 * encontrados en hasta k nodos si el cambio proviene de su soporte, lo que lleva a
 * un mejor rendimiento
```

- * a costa de un mayor tiempo de procesamiento.
- * * * CÓMO LO HACE:
 - * En cada iteración (tick del juego), el agente genera los sucesores inmediatos del estado actual.
 - * Evalúa cada sucesor sumando el coste del paso y su valor heurístico (calculado inicialmente
 - * mediante la distancia Manhattan al objetivo). Selecciona el sucesor con el menor coste total y
 - * si se actualiza el valor heurístico del estado actual se amplía el espacio local de aprendizaje
 - * hasta k nodos sucesores del actual. Los valores heurísticos actualizados se almacenan y
 - * recuperan utilizando una tabla Hash (HashMap).
 - * * * POR QUÉ (Decisiones de diseño y optimización para el rendimiento):
 - * - Uso de HashMap para la heurística y los sucesores: Permite consultar y actualizar los valores
 - * aprendidos de los estados en tiempo constante $O(1)$, minimizando la sobrecarga computacional
 - * durante la búsqueda.
 - * - Variables de estado con BitSet (frente a boolean[][] o listas): El entorno requiere saber
 - * qué monedas, llaves y catapultas siguen activas. Representar esto con matrices implicaría
 - * un alto coste al clonar el estado para simular cada acción. BitSet comprime esta información
 - * a nivel de bits, permitiendo copias y mutaciones casi instantáneas con un consumo de memoria mínimo.
 - * - Evaluación de sucesores "al vuelo": En lugar de instanciar y almacenar temporalmente los
 - * nodos sucesores en estructuras de datos complejas (como listas o colas), los vecinos se
 - * simulan, se evalúan frente a los mejores valores registrados y se descartan inmediatamente.
 - * Esto reduce drásticamente la creación de objetos innecesarios en memoria, aliviando el
 - * trabajo del recolector de basura (Garbage Collector) y acelerando el tiempo total de ejecución.

Respuesta:

/**

* Clase que implementa un agente deliberativo basado en el algoritmo de búsqueda heurística en tiempo real $LRTA^*(k)$.

* * QUÉ HACE:

- * Guía al avatar hacia el objetivo tomando decisiones de movimiento paso a paso (online) entrelazando la búsqueda y la ejecución.
- * A diferencia de los algoritmos offline como A* (que calculan el plan completo a priori), LRTA*(k) determina
 - * la siguiente acción explorando únicamente un espacio local de búsqueda.
 - * * Además, incorpora un espacio local de aprendizaje para evitar quedar atrapado en ciclos infinitos o mínimos locales:
 - * actualiza el valor heurístico de los estados visitados a medida que descubre que el coste real es mayor al estimado inicialmente.
 - * La característica distintiva de la variante parametrizada LRTA*(k) es que propaga estos cambios heurísticos hasta un máximo de k
 - * nodos predecesores, siempre y cuando el cambio provenga de su "soporte" (su mejor vecino), lo que acelera notablemente la
 - * convergencia hacia el camino óptimo en sucesivas ejecuciones.
 - * * CÓMO LO HACE:
 - * En cada ciclo de decisión (tick del juego), el agente realiza los siguientes pasos:
 - * 1. Generación y Evaluación: A partir del estado actual, genera los sucesores inmediatos simulando las acciones permitidas.
 - * 2. Aprendizaje y Propagación (LookaheadUpdateK): Evalúa los sucesores calculando $f(y) = c(x, y) + h(y)$. Aplica la regla
 - * de aprendizaje actualizando la heurística del estado actual con el valor del primer mínimo: $h(x) = \max(h(x), \min(c(x, y) + h(y)))$.
 - * Si $h(x)$ aumenta, el cambio se propaga a los vecinos que lo tienen como soporte (hasta k saltos iterativos usando una cola).
 - * 3. Selección de Movimiento: Selecciona y ejecuta la acción que conduce al vecino con el menor coste estimado (estrategia de movimiento).
 - * * POR QUÉ (Decisiones de diseño y optimización de rendimiento):
 - * Dado el límite estricto de tiempo de procesamiento por tick, se han tomado las siguientes decisiones estructurales:
 - * - Tablas Hash (HashMap) para Heurística y Soportes: Permiten consultar y actualizar los valores aprendidos y la relación de
 - * soportes en tiempo constante $O(1)$, minimizando la sobrecarga computacional durante la actualización del espacio de aprendizaje.
 - * - Variables de estado optimizadas (BitSet frente a boolean[][] o listas): Para representar elementos dinámicos del entorno
 - * (monedas, llaves, catapultas activas) se emplean máscaras de bits. Esto permite que la clonación y mutación del estado al
 - * simular acciones sea casi instantánea y requiera un consumo mínimo de memoria.
 - * - Evaluación de sucesores "al vuelo" (On-the-fly): En lugar de instanciar y almacenar los nodos sucesores en estructuras de
 - * datos intermedias, los estados vecinos se simulan, se evalúan frente a los mejores valores registrados y se descartan o

* sobrescriben inmediatamente. Esto reduce drásticamente la creación de objetos, aliviando el trabajo del Garbage Collector.

*/